

EduWatch AI — Full MVP Structure (FastAPI + PostgreSQL + AI)

Project Goal

Ta'lim tizimidagi shubhali holatlarni aniqlovchi AI monitoring platforma.

Bu MVP quyidagilarni qiladi:

- Universitetlar ro'yxati
- Talabalar ma'lumotlari
- Baholarni saqlash
- AI orqali shubhali baholarni aniqlash
- Complaint (anonim shikoyat)
- Admin dashboard

1. Papkalar Strukturası

```
eduwatch-ai/
|
├── app/
│   ├── main.py
│   ├── database.py
│   ├── models.py
│   ├── schemas.py
│   ├── ai.py
│   ├── crud.py
│   ├── routers/
│   │   ├── university.py
│   │   ├── students.py
│   │   ├── grades.py
│   │   └── complaints.py
│   ├── templates/
│   │   ├── dashboard.html
│   │   └── complaints.html
│   └── static/
│       └── style.css
├── requirements.txt
└── run.py
```

2. requirements.txt

```
fastapi
uvicorn
jinja2
sqlalchemy
psycopg2-binary
pandas
scikit-learn
python-multipart
```

Install:

```
pip install -r requirements.txt
```

3. database.py

Path:

```
app/database.py
```

Code:

```
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker

DATABASE_URL = "postgresql://postgres:password@localhost/eduwatch"

engine = create_engine(DATABASE_URL)

SessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine
)

Base = declarative_base()
```

4. models.py

Path:

```
app/models.py
```

Code:

```
from sqlalchemy import Column, Integer, String, ForeignKey, Float
from sqlalchemy.orm import relationship

from .database import Base

class University(Base):
    __tablename__ = "universities"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String)
    region = Column(String)

class Student(Base):
    __tablename__ = "students"

    id = Column(Integer, primary_key=True, index=True)
    full_name = Column(String)
    university_id = Column(Integer, ForeignKey("universities.id"))

class Grade(Base):
    __tablename__ = "grades"

    id = Column(Integer, primary_key=True, index=True)
    student_id = Column(Integer, ForeignKey("students.id"))
    teacher_name = Column(String)
    subject = Column(String)
    grade = Column(Float)

class Complaint(Base):
    __tablename__ = "complaints"

    id = Column(Integer, primary_key=True, index=True)
    university_name = Column(String)
    text = Column(String)
    status = Column(String, default="new")
```

5. schemas.py

Path:

```
app/schemas.py
```

Code:

```
from pydantic import BaseModel

class UniversityCreate(BaseModel):
    name: str
    region: str

class StudentCreate(BaseModel):
    full_name: str
    university_id: int

class GradeCreate(BaseModel):
    student_id: int
    teacher_name: str
    subject: str
    grade: float

class ComplaintCreate(BaseModel):
    university_name: str
    text: str
```

6. crud.py

Path:

```
app/crud.py
```

Code:

```
from sqlalchemy.orm import Session
```

```
from . import models

def create_university(db: Session, university):
    db_item = models.University(**university.dict())
    db.add(db_item)
    db.commit()
    db.refresh(db_item)
    return db_item

def create_student(db: Session, student):
    db_item = models.Student(**student.dict())
    db.add(db_item)
    db.commit()
    db.refresh(db_item)
    return db_item

def create_grade(db: Session, grade):
    db_item = models.Grade(**grade.dict())
    db.add(db_item)
    db.commit()
    db.refresh(db_item)
    return db_item

def create_complaint(db: Session, complaint):
    db_item = models.Complaint(**complaint.dict())
    db.add(db_item)
    db.commit()
    db.refresh(db_item)
    return db_item
```

7. AI Module

Path:

app/ai.py

Code:

```

import pandas as pd
from sklearn.ensemble import IsolationForest

def detect_suspicious_grades(data):
    df = pd.DataFrame(data)

    model = IsolationForest(contamination=0.1)

    df["anomaly"] = model.fit_predict(df[["grade"]])

    suspicious = df[df["anomaly"] == -1]

    return suspicious.to_dict(orient="records")

```

8. University Router

Path:

```
app/routers/university.py
```

Code:

```

from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session

from ..database import SessionLocal
from .. import schemas, crud

router = APIRouter()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

@router.post("/universities/")
def create_university(
    university: schemas.UniversityCreate,
    db: Session = Depends(get_db)
)

```

```
    ):  
        return crud.create_university(db, university)
```

9. Student Router

Path:

```
app/routers/students.py
```

Code:

```
from fastapi import APIRouter, Depends  
from sqlalchemy.orm import Session  
  
from ..database import SessionLocal  
from .. import schemas, crud  
  
router = APIRouter()  
  
def get_db():  
    db = SessionLocal()  
    try:  
        yield db  
    finally:  
        db.close()  
  
@router.post("/students/")  
def create_student(  
    student: schemas.StudentCreate,  
    db: Session = Depends(get_db)  
):  
    return crud.create_student(db, student)
```

10. Grades Router

Path:

```
app/routers/grades.py
```

Code:

```
from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session

from ..database import SessionLocal
from .. import schemas, crud
from ..models import Grade
from ..ai import detect_suspicious_grades

router = APIRouter()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

@router.post("/grades/")
def create_grade(
    grade: schemas.GradeCreate,
    db: Session = Depends(get_db)
):
    return crud.create_grade(db, grade)

@router.get("/grades/analyze")
def analyze_grades(db: Session = Depends(get_db)):
    grades = db.query(Grade).all()

    data = []

    for g in grades:
        data.append({
            "id": g.id,
            "grade": g.grade,
            "teacher": g.teacher_name,
        })

    return detect_suspicious_grades(data)
```


11. Complaints Router

Path:

```
app/routers/complaints.py
```

Code:

```
from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session

from ..database import SessionLocal
from .. import schemas, crud

router = APIRouter()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

@router.post("/complaints/")
def create_complaint(
    complaint: schemas.ComplaintCreate,
    db: Session = Depends(get_db)
):
    return crud.create_complaint(db, complaint)
```

12. main.py

Path:

```
app/main.py
```

Code:

```
from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles
```

```

from fastapi.templating import Jinja2Templates

from .database import Base, engine
from .routers import university, students, grades, complaints

Base.metadata.create_all(bind=engine)

app = FastAPI(title="EduWatch AI")

app.include_router(university.router)
app.include_router(students.router)
app.include_router(grades.router)
app.include_router(complaints.router)

app.mount("/static", StaticFiles(directory="app/static"), name="static")

templates = Jinja2Templates(directory="app/templates")

@app.get("/")
def home():
    return {
        "message": "EduWatch AI Running"
    }

```

13. dashboard.html

Path:

```
app/templates/dashboard.html
```

Code:

```

<!DOCTYPE html>
<html>
<head>
    <title>EduWatch Dashboard</title>
    <link rel="stylesheet" href="/static/style.css">
</head>
<body>

<h1>EduWatch AI Dashboard</h1>

<div class="card">
    <h2>Risk Monitoring</h2>
    <p>AI suspicious activities monitor</p>

```

```
</div>

<div class="card">
  <h2>Complaints</h2>
  <p>Anonymous complaint system</p>
</div>

</body>
</html>
```

14. complaints.html

Path:

```
app/templates/complaints.html
```

Code:

```
<!DOCTYPE html>
<html>
<head>
  <title>Complaint</title>
</head>
<body>

  <h1>Anonymous Complaint</h1>

  <form>
    <input type="text" placeholder="University name">

    <textarea placeholder="Write complaint"></textarea>

    <button type="submit">Send</button>
  </form>

</body>
</html>
```

15. style.css

Path:

app/static/style.css

Code:

```
body {  
    font-family: Arial;  
    padding: 40px;  
    background: #f2f2f2;  
}  
  
.card {  
    background: white;  
    padding: 20px;  
    margin-top: 20px;  
    border-radius: 10px;  
}
```

16. run.py

Path:

run.py

Code:

```
import uvicorn  
  
if __name__ == "__main__":  
    uvicorn.run(  
        "app.main:app",  
        host="127.0.0.1",  
        port=8000,  
        reload=True  
    )
```

17. PostgreSQL Database Create

Open PostgreSQL:

```
CREATE DATABASE eduwatch;
```

18. Run Project

```
python run.py
```

Open:

```
http://127.0.0.1:8000/docs
```

FastAPI Swagger ochiladi.

19. Future Features

Add JWT Auth

Libraries:

```
pip install python-jose passlib bcrypt
```

Add Redis Queue

```
pip install redis celery
```

Add NLP Complaint AI

```
pip install transformers torch
```

20. Production Deployment

Docker

```
FROM python:3.13

WORKDIR /app

COPY . .

RUN pip install -r requirements.txt

CMD ["python", "run.py"]
```

21. Real Startup Advice

MVP uchun:

- super UI kerak emas
- real problem solving kerak
- demo data bilan boshlang
- 1 ta universitet pilot qiling
- dashboardni chiroyli qiling

B2G'da eng muhimi:

- trust
- analytics
- audit logs
- reporting
- transparency